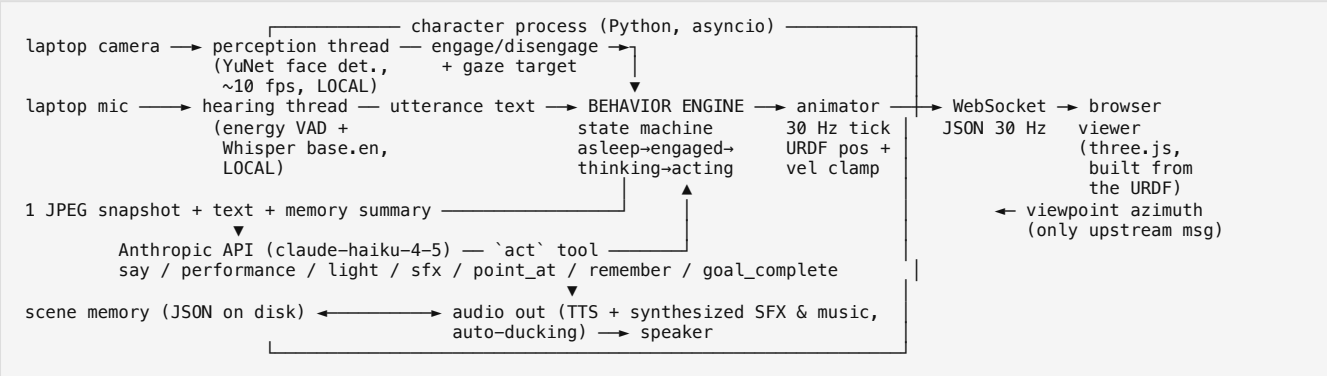
Technical Note: Ikea Lamp, a Live Character Robot

For this challenge I built a live desk-lamp character around the supplied five-degree-of-freedom URDF. The character sees through the laptop camera, listens through the microphone, speaks through the speaker, and expresses itself through motion, light, sound effects, and music in a 3D body rendered in the browser. My primary design goal was that the result should feel like one aware character rather than a collection of separate AI demonstrations, so a single behavior engine owns all state, and every action the character takes is a combined gesture of speech, motion, light, and sound.

1. Architecture and Data Flow



A single Python process owns all state, and the browser acts purely as a display of the robot's body. I treated the URDF as the single source of truth: it is parsed once at startup, the resulting description is sent to the viewer, which constructs the 3D scene from it, and the animator enforces the same file's joint position and velocity limits on every tick. No property of the robot is duplicated by hand anywhere in the codebase.

2. Protocol

Communication uses JSON over a single WebSocket. Downstream messages carry authoritative state: `robot` (sent once on connect with the URDF-derived description), `state` (30 Hz joint positions, hop offset, and light color and intensity), `hud` (behavior state and captions), and `camera` (a 5 Hz annotated webcam thumbnail so that what the character sees is visible to the viewer). There is exactly one upstream message, `viewpoint`, which reports the orbit camera's azimuth; the character treats this as the location of its audience and turns toward it through its real joints. Because the viewer is only a display, the same protocol could drive a physics simulator or real firmware without changes to the character software.

3. Model-to-Action (the VLA Boundary)

For each utterance, Claude receives three inputs: the transcribed text, a single downscaled JPEG snapshot from the camera, and a compact summary of scene memory. It must respond through a forced, strictly validated `act` tool whose schema is the character's complete action vocabulary: `say`, a named motion performance, light color and intensity, a sound effect, an optional `point_at(x, y)` target in image-normalized coordinates, a `remember` list of objects worth storing (name, description, and position), and a `goal_complete` flag. This defines the boundary clearly: the model decides *what* to do in semantic terms grounded in the image, and the controller decides *how*, mapping image coordinates to joint angles and clamping them to the URDF limits. The model never outputs joint values. Goal-directed actions run as a loop in which the character acts, moves, takes a fresh snapshot, and asks Claude to verify its own aim before setting `goal_complete`. Scene memory is a JSON store that Claude writes to through `remember` and reads back through the summary, which allows the character to answer questions such as "where was my mug?" after the object has left the frame.

4. Simulation

I chose kinematic animation rather than a physics engine. The real robot is a desk-mounted servo arm whose dynamics are dominated by its servos, so a physics simulation would add cost and failure modes without adding believability. Physical plausibility is instead enforced where it matters: every tick clamps to the URDF's position and velocity limits (with a 15% derating), and motion is composed from a performance layer of authored keyframes with spring easing, a breathing layer so the character is never perfectly still, and a gaze layer, all summed and then clamped. I selected a custom three.js viewer over PyBullet or MuJoCo because it allowed expressive rendering (a real spotlight, an emissive Edison bulb, a themed body) and because the WebSocket boundary makes the architecture explicit. I took one deliberate liberty: the lamp hops during its "excited" performance. This is implemented as a display-only channel with a ballistic profile, kept separate from the joint state, and a real desk-mounted unit would simply drop it. The URDF itself is unmodified; the viewer only restyles materials and hides the camera-marker geometry.

5. Deployment (Ubuntu 24.04, four cores, 8 GB RAM, no GPU)

Deployment is a plain Python virtual environment plus a small set of `apt` packages, with exact commands in the README; containers were unnecessary at this scale. I sized every CPU-heavy component for the target: YuNet face detection (a 230 KB ONNX model) at roughly 10 fps, faster-whisper `base.en` in int8 (about 0.5 s per utterance on CPU), and a 30 Hz animation loop. Text-to-speech selects a backend automatically, using `piper` (a local neural voice) on Linux and `say` on my macOS development machine. The viewer vendors all of its JavaScript dependencies, so the only network dependency is the Anthropic API. Camera, microphone, and speaker are selectable from the viewer and persisted between runs; cameras are shown as live thumbnails because OpenCV exposes no device names, which mattered in practice when macOS Continuity Camera repeatedly took over camera index 0.

Regarding data sent to the cloud: face detection, voice activity detection, and speech-to-text all run locally. The only data that leaves the machine is the utterance text, one JPEG per exchange, and the memory summary; no audio or video streams are ever transmitted. I chose Claude Haiku over a larger model because reply latency of two to three seconds matters more to a live character than reasoning depth, and the model remains a single configuration setting.

6. Measurements (MacBook development machine, recorded by built-in instrumentation to `metrics.json`)

Metric	Measured
Engagement: face appears to wake begins	~0.8 s (0.7 s debounce plus ~10 fps detection); disengage after 2.5 s of absence. No false wakes over multi-minute idle runs and no flicker, since both transitions are debounced
Speech-to-text latency (utterance end to text)	**0.45 s** (base.en int8 on CPU)
LLM latency (snapshot and text to action)	**2.6–2.9 s** (claude-haiku-4-5, forced tool call)
Utterance end to reply start	≈ 3.5 s (STT, LLM, and gesture kickoff)
Utterance end to reply finished	7.7 s measured; the remainder is TTS speaking a one- to two-sentence reply (2.8–4.8 s)
CPU, whole process, engaged and tracking	**~60% of one core** (peak 74%)
Memory (RSS)	**~700 MB** (peak 860 MB), dominated by Whisper, OpenCV, and numpy audio buffers; well within the 8 GB target
Camera pipeline	10.5 fps sustained; animation at 30 Hz

The LLM call is the dominant source of latency. Streaming a text-first response would reduce perceived latency, but it would break the single-atomic-action contract described above, so I kept the design and instead masked the wait with a "thinking" pose and a pulsing light.

7. Timebox

I spent about seven hours across two days, using AI-assisted development throughout. The architecture, behavior design, measurements, and technical decisions in this note are my own.

8. Known Limitations and Intentional Scope Cuts

- Aiming uses a fixed camera-to-joint mapping (treating the laptop camera as the lamp's forward view) rather than calibrated hand-eye geometry; verification by re-observation compensates for coarse aim.
- Engagement is based on face presence. YuNet's bias toward frontal faces approximates "looking toward the character," but there is no true gaze estimation, and the system handles a single person with no face identification across sessions.
- The energy-based voice activity detector performs well in normal rooms, and its asymmetric floor tracking absorbs fans and the character's own music bed, but very noisy environments would call for a neural VAD such as Silero, which I cut for scope.
- There is no physics or collision simulation and no inverse kinematics; expressive poses are authored and pointing uses a two-joint mapping. This is appropriate for a lamp but would not scale to a manipulator.
- The Ubuntu setup instructions are written and dependency-checked but were not run on a clean 24.04 machine; development and measurement were done on macOS.